

Class 12: Agile Toolkit for Data and ML Teams

Data-driven Systems Engineering

Agenda

- Collaboration and communication
- Development practices
- Planning and tracking
- Agile frameworks and roles
- Feedback and customer involvement
- Agile anti-patterns in data and ML teams

Agile practices are only useful if they change behavior

Learning goals

- Use ceremonies and artifacts as coordination tools, not rituals.
- Adapt user stories, estimation, and boards to data-intensive projects.
- Connect agile practices to quality, deployment, and stakeholder learning.
- Recognize where standard agile templates break down in exploratory ML work.

Course Map and Running Cases



Today in the course

Focus: Day-to-day agile coordination for data and ML work.

Bridge: This class turns process-model ideas into concrete planning, review, and feedback practices that teams can actually use.

Fraud case

In the fraud case, agile work means slicing the backlog across data checks, model tuning, API behavior, and validation evidence.

Pasteurization case

In the pasteurization case, agile work means making sensor issues, drift tasks, dashboard improvements, and monitoring criteria visible on the board.

Course thread: The thread is that agile only helps when it improves coordination across the full ML lifecycle rather than only speeding up coding.

Agile toolkit means a set of coordination mechanisms

- Meetings, boards, stories, and metrics are not the goal of agility.
- Their value comes from making priorities visible, reducing blockers, and shortening feedback loops.
- In ML teams, they also help coordinate data work, experimentation, software integration, and operations.

Which agile tools fit ML projects best?

- Sprint planning for defining a thin vertical slice.
- Backlog refinement for clarifying data dependencies and acceptance criteria.
- Daily stand-ups for surfacing blockers such as missing labels or broken environments.
- Retrospectives for process improvement, especially around experimentation waste.

Collaboration rituals and what they actually solve

Practice	Practical value
Sprint planning	Aligns ambition with capacity and dependencies.
Daily stand-up	Surfaces blockers before they become schedule failures.
Backlog refinement	Prevents ambiguous work from entering a sprint.
Retrospective	Improves the way the team works, not only what it built.
Review or demo	Tests whether the increment creates stakeholder value.

Development practices that matter most

- Test-driven development for stable service behavior.
- CI for repeated checks on code and packaging.
- CD for safe release preparation.
- Pair review for notebooks, requirements, and production code.
- Retrospectives for process correction, not blame.

Development practices for notebook-heavy teams

- Review notebooks for reproducibility, not only for narrative clarity.
- Move repeated feature logic into modules before it spreads across files.
- Treat data validation and schema checks as quality work, not optional cleanup.
- Use small pull requests so code review remains meaningful.

Course link

This is where agile meets the software engineering and MLOps parts of the course: speed without quality creates faster confusion.

Better user stories for this course

Weak

As a student, I want an AI model.

Better

As a quality engineer, I want to detect abnormal temperature trajectories during HeatUp so that I can inspect the pasteurization line before product quality drops.

A better user story also needs acceptance criteria

Story

As a quality engineer, I want abnormal HeatUp trajectories flagged so that I can inspect the pasteurization line before product quality drops.

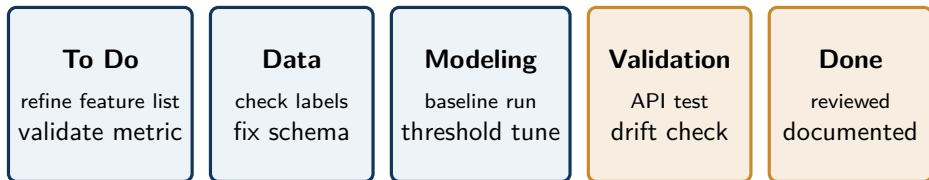
Acceptance criteria

- The system ingests live or simulated sensor points.
- It displays the current state and a risk flag.
- Alerts are logged with timestamp and model version.
- A false-positive threshold is agreed with stakeholders.

Planning and tracking tools

- Kanban boards for work visibility and WIP control.
- User stories with acceptance criteria.
- Story points for relative complexity, not exact duration.
- Burndown and velocity as team learning tools, not performance theater.

A Kanban board adapted to ML work



- This kind of board makes hidden data work visible instead of pretending every task is pure coding.

Planning tools students can actually use

- Kanban board with columns for data, modeling, integration, and validation.
- Story points only after the team shares a definition of complexity.
- Burndown charts for sprint scope, not for pretending uncertainty does not exist.
- Definition of Ready and Definition of Done for every deliverable.

Estimation is for coordination, not prediction theater

- Story points work best as a relative complexity language shared by the team.
- Estimation fails when people use points as disguised hours.
- High uncertainty tasks should often be split into investigation work before full implementation.
- Velocity should guide planning, not evaluate people.

Agile meets DevOps and MLOps

- Continuous integration protects the code path.
- Continuous delivery protects the release path.
- Monitoring closes the loop after release.
- Without operational feedback, agile becomes local optimization.

Structured rhythm

- Scrum for bounded sprint goals
- Good for student teams needing cadence
- Useful when review moments matter

Flow orientation

- Kanban for continuous incoming work
- Good for ops-heavy or support-heavy teams
- Useful when work types vary a lot

Bridge

DevOps and MLOps extend both by asking how work is built, shipped, observed, and improved after release.

- Scrum: structured events, roles, and sprint rhythm.
- Kanban: flow-oriented work management.
- DevOps: shared responsibility across build, release, and operations.
- MLOps: extends DevOps with data, models, drift, and governance concerns.

Agile anti-patterns in data teams

- Treating research uncertainty as if it were ordinary feature implementation.
- Filling sprints with vaguely defined “improve model” tasks.
- Ignoring data dependencies until late integration.
- Using demos that show only happy-path notebooks instead of operational evidence.
- Running retrospectives that identify problems but never change behavior.

What good agile behavior looks like in this course

Repo activity

- Use notebooks to explore, then promote reusable logic into scripts or services.
- Demonstrate thin slices such as ingest plus baseline plus API rather than isolated fragments.
- Keep project boards aligned with requirements, deployment work, and monitoring plans.
- Show how feedback changes backlog priorities instead of pretending the first plan was complete.

From This Class to the Next

What stays with us

- Agile in data-driven work means learning quickly while preserving evidence and alignment.
- Stories, boards, demos, and retrospectives are useful only when they reflect technical and data reality.
- Thin vertical slices are stronger than isolated notebook achievements.

Project use

Teams should now be able to organize milestones, backlog items, and reviews around deployable progress instead of vague promises to improve the model later.

Next connection

Class 13 reconnects these practices to MLOps, where experimentation, delivery, ownership, and monitoring become one operational system.

Course thread: Agile coordination gives the team a way to move continuously; MLOps gives that movement technical reliability.

Papers and materials

Scrum Guide; What is Kanban; Learning Agile; Martin Fowler on user stories; Forsgren, Humble, Kim, Accelerate.